Josh Kindarara
Daryl Ho

Evaluating Scalability and Latency in Distributed Hash Tables

The Problem

As distributed systems scale to accommodate increasing numbers of users, data, and resources, one of the challenges that becomes quickly apparent is scalability. Traditional client-server architectures struggle to efficiently manage growing workloads, since a centralized server becomes a bottleneck once the load becomes too large. When the number of nodes and requests grow, maintaining consistent performance and reliability through a server becomes increasingly difficult. This limitation has motivated many to come up with a decentralized approach to distributed systems, where no single node holds complete responsibility.

In large-scale decentralized systems, efficiently locating and retrieving data without relying on a centralized server remains a fundamental challenge. Many client-server architectures struggle with scalability, single points of failure, and load management [1]. Distributed Hast Tables (DHTs) address this problem by providing a fully decentralized mechanism for key-based lookup across a dynamic network of nodes.

In a DHT, data is distributed among participating nodes using consistent hashing so that each node is responsible for a portion of the key space. When a node joins or leaves, the system must rebalance keys such that the routing remains correct.

Motivation

DHTs are used as the infrastructure for many large, complex online services and applications. It is widely utilized for file sharing, storage, and even blockchain. However, while traditional DHTs like Chord have been elegant solutions, they do not consider network proximity or latency in their routing decisions. This limitation leads to poor user experience in geographically diverse networks.

This project aims to build a bridge between theoretical distributed algorithms and practical performance by applying distributed system concepts to a real-world use case. Challenges may arise as this project requires implementing routing algorithms that adapt dynamically to node churn, as nodes join, leave, and fail. Another challenge that is commonly faced with DHTs is maintaining relatively strong stability to fix inconsistencies introduced by churn. A balance must be carefully found between stability and performance.

This project should benefit developers seeking to build decentralized applications, such as file sharing applications. It should also benefit system designers exploring large, scalable, fault-tolerant networks and alternatives to current solutions.

Previous Work

Before the concept of DHTs, the beginning of peer-to-peer (P2P) systems started with Napster’s implementation of a widely-deployed P2P system. Napster was a website used to host

a large amount of song files where users could upload files and share them. Napster's strategy was to utilize peers that stored the files on their local machines and a server that stored the locations of the files in tuples. This way, the bottleneck of the central site would be removed and the server would not be responsible for any file transmission to clients. Unfortunately, this does not decentralize the system, as the server is still a single point of failure; once the server fails, the system loses all ability to locate the files. However, the idea of distributed file storage was adopted into the more decentralized method of DHTs.

The limitations of a system like Napster highlight the need for a decentralized system, one that could remove the reliance on centralized servers while still efficiently locating data across a large number of peers. This need gave rise to DHTs, which introduced structured P2P systems with scalable, fault-tolerant data lookup.

DHTs are known for having efficient O(log N) lookups. However, real-world DHTs face challenges, especially for large networks. Chord uses a circular, one-dimensional ring where nodes are ordered by their IDs. A newer algorithm, Kademlia, was later introduced. It uses a multi-dimensional space where "distance" is defined by the bitwise XOR of node IDs. This creates a symmetric and more flexible network structure, causing Kademlia to have higher fault tolerance and more efficient routing.

Even as the number of peers increase with Kademlia, the lookup latency and hop count increase as well. For a large-scale network, this would become a huge issue. KadRTT is an alternative to Kademlia that "reduces both the lookup latency and hop count," making it a more efficient algorithm [2].

More recent research has revisited Kademlia's use in production systems such as IPFS and Ethereum [3]. One recent study proposed a next-generation DHT framework that combines latency-aware routing, secure network coordinates, and trusted execution environments (TEE) to address the growing performance and security needs of web applications. The researchers simulated both vanilla Kademlia and Ethereum's modified version across a large number of nodes. They found that minor routing changes such as withholding query keys improved both latency and resistance to Eclipse attacks. This work demonstrates the measurable performance benefits that can result, motivating our intent to build and test an efficient DHT in a simulated environment.

Initial Ideas

For our implementation, we plan to use C++ and leverage C's socket library for asynchronous communication between nodes. We plan to use the well-established ring chord topology, utilizing SHA-1 for consistent hashing on each node's IP address and port. Each node will be assigned a peer_id based on its SHA-1 hashing result, truncated to m bits. From there, a ring topology can be created where each node's successors have sequentially larger peer_ids. After the ring topology is established, the finger table for each node can be calculated, ensuring that each node in a ring of N nodes will only see at most log(N) successors.

We aim for O(log N) runtime for lookups and routing, and $O(N^2)$ total stabilization cost during large-scale churn scenarios, consistent with Chord's expected time complexity. Our eventual goal is to simulate and visualize how network size affects lookup latency. Inspired by Sokoto et al. [3], we may incorporate simulation-based testing to compare performance metrics under controlled environments similar to their evaluation of Ethereum's DHT.

To run and evaluate the system, we plan to:

- Simulate multiple nodes locally on a single machine using Docker container or lightweight processes.
    - Each container will emulate a separate DHT node.
- Use Docker Compose to scale the number of nodes dynamically for different experiments.
- Deploy a small cluster on AWS EC2 or another cloud provider to observe real network latency effects and compare them to local simulations.
- Run scripted experiments that:
    - Measure average lookup latency as the number of nodes increases.
    - Simulate random node churn to analyze stabilization behavior.
    - Record throughput and load balance across the network.

These tests will help us evaluate how well the proposed optimizations performed.

The Plan

Our first stage of our project will mainly focus on developing the foundational components of a Chord-based DHT. During this stage, we will implement the basic node module and consistent hashing mechanism, creating the system of assigning peer_ids and file mappings, and allowing nodes to store and retrieve key-value pairs through standard get() operations. Once we have the core functionality working, we will extend the system to allow for node joining and leaving mechanisms, alongside consistent updates to finger tables and implementing the stabilization protocol.

After achieving a functional Chord implementation, the next stage will involve testing and performance evaluation of our implementation of Chord. We will conduct controlled tests on AWS EC2 in order to observe and measure the system's stability, scalability, and look-up latency under varying conditions, such as network size and churn rate. The metric we will be focusing on is how the protocol adapts to an increase in the number of nodes, as our emphasis is on scalability. This, along with latency, will be our performance baseline for comparison with the other DHT protocol.

Once we have finished exhaustive testing with Chord, we will move to implement the Kademlia DHT, where its implementation will differ in routing strategy and look-up procedures. However, our approach will remain the same; implement the basic features first, then moving to more complicated issues like node joining/leaving, and then performance testing under varying scenarios. We will conclude the project by conducting a comparative analysis on Kademlia and

Chord using the performance metrics we established, writing a report that summarizes our findings, and creating a presentation of our entire project process, implementation, and results.

Resources Required

To develop, simulate, and test our DHT implementation, we will use the following tools and environments:

- C++ - for implementing the core DHT logic
- Docker containers - for emulating multiple nodes locally
- Docker Compose - for easily scaling node count and managing network configuration
- AWS EC2 - for cloud-based experiments involving real network latency

Citations

[1] H. Huang, H. Wu and G. Wang, "Study of Distributed P2P Information Sharing System," 2009 Third International Symposium on Intelligent Information Technology Application, Nanchang, China, 2009, pp. 300-303.

[2] H. Kanemitsu and H. Nakazato, ‘KadRTT: Routing with network proximity and uniform ID arrangement in Kademlia’, in 2021 IFIP Networking Conference (IFIP Networking), 2021, pp. 1–6.

[3] S. Sokoto, M. Król, V. Stankovic, and E. Rivière, ‘Next-Generation Distributed Hash Tables’, in Proceedings of the on CoNEXT Student Workshop 2023, Paris, France, 2023, pp. 29–30.

[4] A. Rawat, A. Gupta and R. Gupta, "Chord Protocol: Distributed Hash Table for Peer-to-Peer Networking," 2024 7th International Conference on Contemporary Computing and Informatics (IC3I), Greater Noida, India, 2024, pp. 727-733.